==========

==========

CHAPTER 17 — MEMORY ORDERING AND BARRIERS (ADVANCED)

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of ARMv8's advanced memory ordering rules, multi-core consistency, device ordering semantics, and the precise role of DSB/DMB/ISB in enforcing architectural guarantees.

****

****

17.1 — Introduction to Advanced Memory Ordering

****

****

Memory ordering defines **when** memory operations become visible to:

- the same core
- other cores
- caches
- TLBs
- DMA engines
- devices
- hypervisors

ARMv8 uses a **weakly ordered memory model**, meaning:

- loads/stores may reorder
- speculation may occur
- prefetch may occur
- write buffers may delay visibility
- device memory ordering must be enforced manually

This chapter explains the **advanced ordering rules** that govern all memory interactions.

****

****

17.2 — ARMv8 Memory Model Overview

****

****

ARMv8 uses a **weakly ordered** model:

- loads may reorder with loads
- stores may reorder with stores
- loads may reorder with stores
- stores may reorder with loads
- speculation may reorder operations

- prefetch may reorder operations

Exceptions

- Device memory is strongly ordered
- Barriers enforce ordering
- Shareability enforces coherence
- TLB invalidation enforces translation ordering

****

****

17.3 — Ordering Domains

****

****

Ordering applies across:

Local Domain

- within the same core
- pipeline ordering
- store buffer ordering

Inner Shareable Domain

- within the same cluster
- L1/L2 coherence
- TLB broadcast

Outer Shareable Domain

- across clusters
- L3 coherence

System Domain

- devices
- DMA engines
- interconnect

****

****

17.4 — Ordering of Normal Memory

****

****

Normal memory is:

- weakly ordered
- cacheable
- speculative
- prefetchable

Allowed reorderings

- Load → Load
- Store → Store
- Load → Store
- Store → Load

Forbidden reorderings

Only when barriers are used.

****

****

17.5 — Ordering of Device Memory

****

****

Device memory is:

- strongly ordered
- non-cacheable
- non-speculative
- non-prefetchable

Forbidden reorderings

- Load → Load
- Store → Store
- Load → Store
- Store → Load

Device memory ordering is enforced by:

- MAIR attributes
- shareability = Non-Shareable
- DSB barriers

****

****

17.6 — Ordering of Atomic Operations

****

****

Atomic operations require:

- Normal memory
- Inner Shareable
- cacheable attributes

Atomic operations enforce:

- acquire semantics
- release semantics
- full ordering when required

Illegal

Atomic operations on:

- Device memory
- Non-Shareable memory
- Non-cacheable memory

****

****

17.7 — Ordering of TLB Operations

****

****

TLB invalidation ordering requires:

Code

```
TLBI
DSB SY
ISB
```

Why?

- TLBI must complete globally
- DSB ensures completion
- ISB ensures pipeline sees new translations

Skipping any step causes **silent corruption**.

****

****

17.8 — Ordering of Page Table Updates

****

****

Correct ordering:

1. Write descriptor
2. **DSB SY**
3. **TLBI**
4. **DSB SY**
5. **ISB**

Why?

- descriptor must be visible
- stale TLB entries must be removed
- pipeline must flush stale state

****

****

17.9 — Ordering of Device Register Access

****

****

Device registers require strict ordering.

Write ordering

```
STR x0, [MMIO]
DSB SY
```

Read ordering

```
DSB SY
LDR x0, [MMIO]
```

Why?

- device may malfunction if writes reorder
- device may return stale data if reads reorder

****

****

17.10 — Ordering of DMA Operations

****

****

DMA engines operate outside CPU caches.

Before DMA Read

```
DC CVAC, buffer
DSB SY
```

After DMA Write

```
DC CIVAC, buffer
DSB SY
```

Why?

- CPU writes must reach memory before DMA reads
- DMA writes must reach memory before CPU reads

****

****

17.11 — Ordering of Speculation

****

****

Speculation may reorder:

- loads
- stores
- instruction fetches
- TLB accesses
- page table walks

Barriers prevent speculative reordering.

ISB

Flushes pipeline.

DSB

Prevents speculative memory access.

DMB

Prevents speculative ordering.

****

****

17.12 — Ordering of Prefetch

****

****

Prefetch may reorder:

- loads
- stores
- device accesses (if misconfigured)

Rules

- Prefetch allowed for Normal memory
- Prefetch forbidden for Device memory

Incorrect prefetch causes device malfunction.

****

****

17.13 — Ordering of Stage-1 vs Stage-2

****

****

Stage-1 ordering applies to:

- VA → IPA translation
- guest memory accesses

Stage-2 ordering applies to:

- IPA → PA translation
- hypervisor memory accesses

Combined ordering

Hypervisor overrides guest ordering.

****

****

17.14 — Ordering of Shareability Domains

****

****

Inner Shareable

- ordering enforced across cores
- required for SMP correctness

Outer Shareable

- ordering enforced across clusters

Non-Shareable

- ordering enforced only within core
- used for device memory

****

****

17.15 — Ordering and MAIR Attributes

****

****

MAIR defines:

- cacheability
- ordering
- device semantics

Normal memory

Weakly ordered.

Device memory

Strongly ordered.

****

****

17.16 — Ordering and TCR Attributes

****

****

TCR defines:

- shareability
- cacheability
- granule size

TCR ordering applies only to:

- page table walks
- not final memory accesses

****

****

17.17 — Ordering and TTBR Attributes

****

****

TTBR defines:

- inner/outer cacheability
- shareability

TTBR ordering applies only to:

- page table walks
- not final memory accesses

****

****

17.18 — Ordering and SMP Systems

****

****

SMP systems require:

- Inner Shareable memory
- DSB SY for global visibility
- broadcast TLBI
- consistent TCR/MAIR configuration

Common SMP ordering bugs

- mapping RAM as Non-Shareable
- missing DSB after TLBI
- missing ISB after MMU changes

****

****

17.19 — Ordering and QNX/RTOS Behavior

****

****

QNX requires:

- deterministic ordering
- strict barrier usage
- strict device ordering
- strict DMA ordering
- strict SMP ordering

QNX uses:

- Inner Shareable for RAM
- Non-Shareable for MMIO
- deterministic barrier sequences

****

****

17.20 — Full Ordering Example

****

****

Example: Updating a page descriptor safely

```
STR x0, [descriptor]        // write new descriptor
DSB SY                      // ensure visibility
TLBI VAE1, x1               // invalidate TLB entry
DSB SY                      // ensure TLBI complete
ISB                         // flush pipeline
```

Result

- descriptor visible
- stale TLB entries removed
- pipeline updated
- ordering guaranteed

****

****

17.21 — Chapter Summary

****

****

This chapter covered:

- ARMv8 weak memory model
- ordering domains
- normal vs device ordering
- atomic operation ordering
- TLB ordering
- page table update ordering
- device register ordering
- DMA ordering
- speculation ordering
- prefetch ordering
- Stage-1 vs Stage-2 ordering
- shareability ordering
- MAIR/TCR/TTBR ordering
- SMP ordering
- QNX/RTOS ordering
- full examples